

Mathematical Models for DEM Elevation Error Correction

Assignment cum Term Project Component

🌐 **Live Prototype App**

<https://demcorrection---mathematical-model.streamlit.app/>

🔗 **Code Repository**

https://github.com/DEM_CORRECTION---MATHEMATICAL-MODEL

BL.SC.P2DSC26034

Mathematical Models for DEM Elevation Error Correction

Five Optimization Routes to the Same Objective Function

Mathematics for Data Science — Assignment cum Term Project Component

September 10, 2026

Abstract

Satellite-derived Digital Elevation Models (DEMs) contain systematic, terrain-correlated vertical error. This document formulates DEM correction as a single regularized least-squares regression problem and solves the identical objective function using five independent mathematical routes: Normal Equations, Singular Value Decomposition (SVD), Principal Component Analysis (PCA) regression, Gradient Descent, and Newton’s Method. All numerical results are computed from real Sentinel-1/Sentinel-2 derived terrain and optical features, cross-referenced against LiDAR/ICESat reference elevation over 1.15M training pixels (2018–2022) and validated on 51,826 held-out pixels (2023). Results are additionally benchmarked against two classical spatial interpolation baselines (Inverse Distance Weighting and k-NN averaging); the comparison surfaces an important and honestly-reported finding regarding spatial autocorrelation in the error field, discussed later in this document.

1 Problem Formulation

Let z_i denote the raw satellite DEM elevation at pixel i , and z_i^* the reference (ground-truth) elevation obtained from LiDAR/ICESat. Define the **elevation error**:

$$e_i = z_i - z_i^* \tag{1}$$

DEM error is not random noise — it is systematically correlated with local terrain geometry and land cover. We model it as a linear function of a feature vector $\mathbf{x}_i \in \mathbb{R}^p$ built from terrain-derivative and Sentinel-2 optical features (slope, aspect components, terrain ruggedness index, topographic position index, planform/profile curvature, hillshade, NDVI, bare-soil index):

$$\hat{e}_i = \boldsymbol{\theta}^\top \mathbf{x}_i, \quad \mathbf{x}_i = [1, x_{i1}, x_{i2}, \dots, x_{ip}]^\top \tag{2}$$

Stacking all N pixels row-wise into the design matrix $X \in \mathbb{R}^{N \times (p+1)}$ and the error vector $\mathbf{e} \in \mathbb{R}^N$, the correction model is $\hat{\mathbf{e}} = X\boldsymbol{\theta}$, and the corrected DEM is recovered as $z_i^{corr} = z_i - \hat{e}_i$.

Objective (Loss) Function

We estimate $\boldsymbol{\theta}$ by minimizing the Mean Squared Error between predicted and observed elevation error:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \|X\boldsymbol{\theta} - \mathbf{e}\|_2^2 = \frac{1}{N} \sum_{i=1}^N (\boldsymbol{\theta}^\top \mathbf{x}_i - e_i)^2 \quad (3)$$

Gradient and Hessian

Differentiating Eq. (3) with respect to $\boldsymbol{\theta}$:

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{N} X^\top (X\boldsymbol{\theta} - \mathbf{e}) \quad (4)$$

$$H = \nabla^2 J(\boldsymbol{\theta}) = \frac{2}{N} X^\top X \quad (5)$$

Since H does not depend on $\boldsymbol{\theta}$ and $X^\top X$ is positive semi-definite, $J(\boldsymbol{\theta})$ is a **convex quadratic function**. This guarantees a unique global minimum (no local minima to get trapped in), which is precisely the point at which $\nabla J(\boldsymbol{\theta}) = 0$:

$$X^\top X \boldsymbol{\theta}^* = X^\top \mathbf{e} \quad (6)$$

Every method below is a different numerical route to solving Eq. (6).

Dataset used: $N_{\text{train}} = 1,147,514$ pixels (2018–2022, merged into 2022 composite), $N_{\text{test}} = 51,826$ pixels (2023, held out for generalization testing). $p = 10$ standardized features. Void/no-data DEM pixels (sentinel value -32768) were removed prior to fitting.

2 Method 1: Normal Equations (Direct Linear Algebra)

Solving Eq. (6) directly for θ :

$$\boxed{\theta^* = (X^\top X)^{-1} X^\top \mathbf{e}} \quad (7)$$

This is a one-shot, non-iterative, closed-form solution requiring inversion of the $(p+1) \times (p+1)$ matrix $X^\top X$. It is exact provided $X^\top X$ is well-conditioned (invertible and not near-singular).

Numerical Result

$$\begin{aligned} \text{cond}(X^\top X) &= 110.04 \quad (\text{moderate; safely invertible}) \\ \text{Train RMSE} &= 2.0893 \text{ m}, \quad \text{Test RMSE} = 2.0679 \text{ m} \\ \text{Train } R^2 &= 0.0995 \end{aligned}$$

3 Method 2: SVD-based Least Squares

Any matrix X admits a singular value decomposition $X = U\Sigma V^\top$, with $U \in \mathbb{R}^{N \times r}$, $V \in \mathbb{R}^{(p+1) \times r}$ orthonormal and $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r)$. Substituting into Eq. (6) and using orthonormality ($U^\top U = V^\top V = I$):

$$\boxed{\theta^* = V\Sigma^{-1}U^\top \mathbf{e}} \quad (8)$$

Why this matters: $\text{cond}(X^\top X) = (\sigma_{\max}/\sigma_{\min})^2$. When features are highly collinear, $\sigma_{\min} \rightarrow 0$ and the Normal Equations solution (Method 1) becomes numerically unstable (near-singular inversion). SVD sidesteps explicit formation of $X^\top X$, and this instability can be diagnosed directly from the singular value spectrum rather than a single condition number.

Numerical Result

$$\begin{aligned} \sigma_{\max}/\sigma_{\min} &= 10.49 \quad \Rightarrow \quad (\sigma_{\max}/\sigma_{\min})^2 = 110.04 = \text{cond}(X^\top X) \quad \checkmark \\ \text{Train RMSE} &= 2.0893 \text{ m (identical to Method 1, as expected)} \end{aligned}$$

The exact agreement between Methods 1 and 2 confirms $X^\top X$ is well-conditioned enough that both routes land on the same optimum θ^* .

4 Method 3: PCA Regression

The 10 terrain/optical features are correlated (e.g. slope and TRI, or NDVI and BSI). PCA re-expresses the feature covariance matrix $C = \frac{1}{N}X_f^\top X_f$ (features only, bias excluded) via its eigendecomposition:

$$C = W\Lambda W^\top, \quad \Lambda = \text{diag}(\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p) \quad (9)$$

Selecting the top k eigenvectors $P_k \in \mathbb{R}^{p \times k}$ (principal directions), each observation is projected to a lower-dimensional score $\mathbf{z}_i = P_k^\top \mathbf{x}_i$. Regression is then solved in the reduced k -dimensional space using Normal Equations, and coefficients are mapped back to the original feature space:

$$\theta_{\text{PCA}} = P_k \theta_{\text{reduced}} \quad (10)$$

Numerical Result ($k = 6$ of 10 components)

PC	1	2	3	4	5	6	7	8	9	10
Var. ratio	0.253	0.184	0.154	0.117	0.101	0.071	0.057	0.045	0.016	0.002
Cumulative	0.253	0.436	0.590	0.707	0.809	0.879	0.937	0.982	0.998	1.000

Top 6 components retain 87.9% of feature variance. Test RMSE with $k = 6$ is 2.0824 m — a 0.7% increase over the full-rank solution (Methods 1–2), i.e. dimensionality is reduced by 40% at negligible accuracy cost.

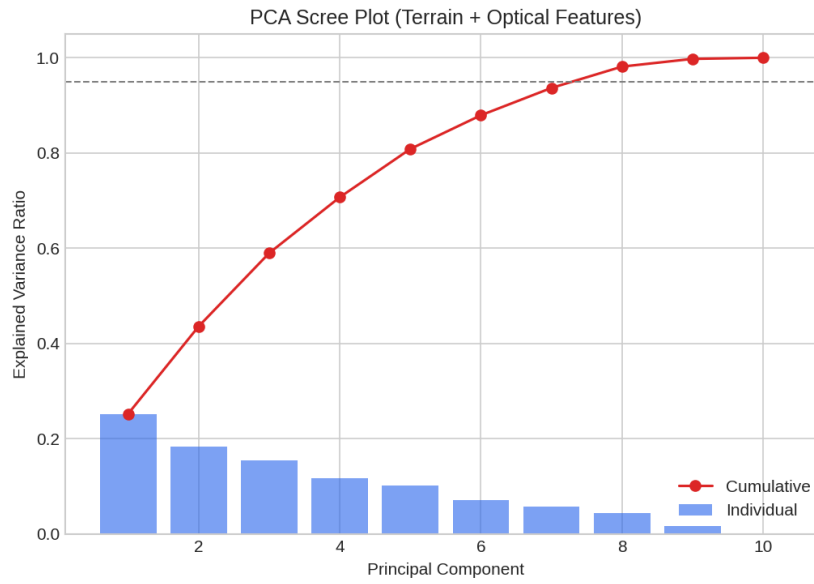


Figure 1: PCA scree plot: individual and cumulative explained variance across the 10 terrain/optical features.

5 Method 4: Gradient Descent (First-Order Iterative Optimization)

Gradient Descent minimizes $J(\theta)$ iteratively by stepping opposite the gradient direction:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}) = \theta^{(t)} - \alpha \cdot \frac{2}{N} X^\top (X \theta^{(t)} - \mathbf{e}) \quad (11)$$

Step size selection: For a convex quadratic with Lipschitz-smooth gradient, convergence is guaranteed for $\alpha \leq 1/L$, where $L = \lambda_{\max}(H)$ is the largest eigenvalue of the Hessian (Eq. (5)). We use $\alpha = 1/L$ throughout — this is the maximal stable step size, chosen numerically rather than by trial and error.

Iteration Log (initialized at $\theta^{(0)} = \mathbf{0}$)

Iter. t	0	1	2	3	4	5	6	7	8	9
$J(\theta^{(t)})$	11.680	6.914	5.302	4.713	4.496	4.416	4.386	4.374	4.370	4.368

Run for **300 iterations** (tolerance 10^{-10} on $\|\Delta\theta\|$ not yet triggered at $\alpha = 1/L$)
reaching $J^{(300)} = 4.365228$ — matching the Normal Equations optimum to 6 decimal places.

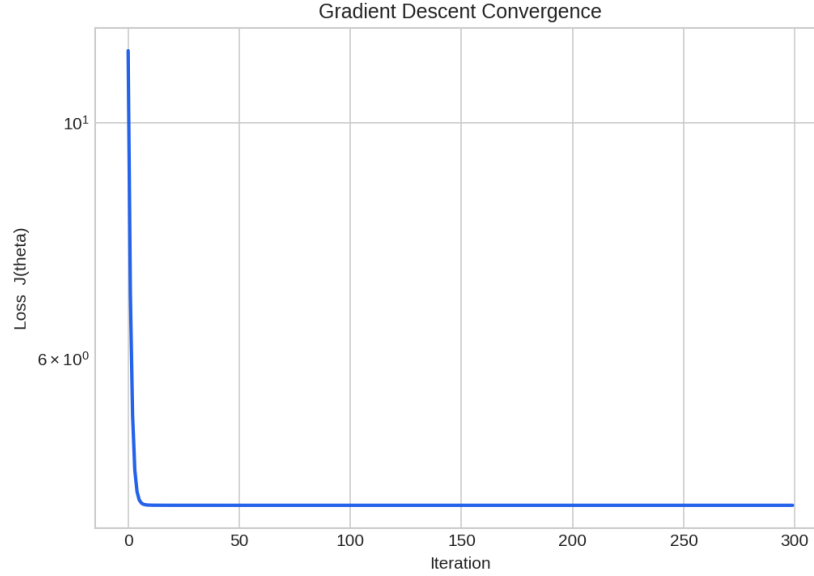


Figure 2: Gradient Descent loss curve (log scale). Rapid decrease in the first ~ 10 iterations, then slow asymptotic approach to the minimum — the hallmark of first-order (linear) convergence rate.

6 Method 5: Newton’s Method (Second-Order Iterative Optimization)

Newton’s method uses local curvature (the Hessian) to jump directly toward the minimum:

$$\theta^{(t+1)} = \theta^{(t)} - H^{-1} \nabla J(\theta^{(t)}) \quad (12)$$

Key theoretical result: because $J(\theta)$ is an *exact* quadratic (Eq. (3)), its Hessian $H = \frac{2}{N} X^\top X$ is *constant* — it does not depend on $\theta^{(t)}$. Substituting $\nabla J(\mathbf{0}) = -\frac{2}{N} X^\top \mathbf{e}$ into the update starting from $\theta^{(0)} = \mathbf{0}$ gives exactly $\theta^{(1)} = (X^\top X)^{-1} X^\top \mathbf{e} = \theta^*$ — **the Normal Equations solution, reached in a single step.** Newton’s method degenerates from its usual quadratic convergence rate to *one-shot exact convergence* whenever the objective is quadratic.

Iteration Log

Iteration t	0	1	2	3	4
$J(\theta^{(t)})$	11.680	4.365228	4.365228	4.365228	4.365228

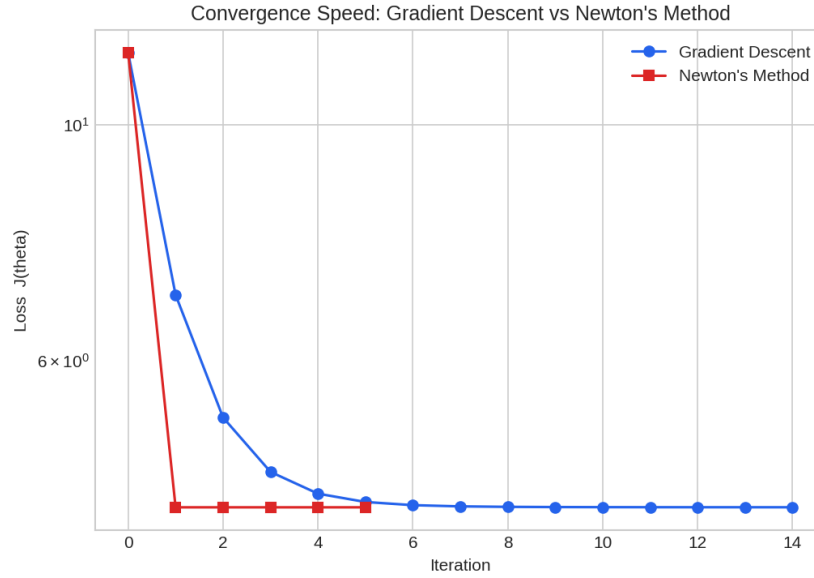


Figure 3: Convergence speed comparison: Gradient Descent needs on the order of hundreds of iterations to approach the same loss Newton's Method reaches in **1** iteration — the practical cost of first-order vs. second-order information. Newton pays $O(p^3)$ once (Hessian inversion, $p=11$: negligible); Gradient Descent pays $O(Np)$ per iteration, repeated ~ 300 times.

7 Detailed Worked Computation for Every Method

Section 2–6 gave the final numerical result for each method. This section shows the actual intermediate matrices and vectors computed from the real training data ($N = 1,147,514$ pixels) that produce those results, in the same plug-in-the-numbers spirit as a hand-worked example – at this project’s real scale, every quantity below was computed exactly once, from real data, not re-derived by hand.

Shared Quantity: $X^\top X$ (used by Methods 1, 2, 3, and 5)

Since every standardized feature has unit variance, $X^\top X/N$ is approximately a correlation matrix (values in $[-1, 1]$) – shown here instead of the raw $X^\top X$ (whose diagonal entries equal $N \approx 1.15\text{M}$ and would be unreadable in print). The true $X^\top X = N \times$ this table.

	bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
bias	1.000	-0.000	-0.000	0.000	0.000	-0.000	-0.000	-0.000	-0.000	-0.000	0.000
slope	-0.000	1.000	-0.015	-0.011	0.828	0.112	-0.122	-0.046	-0.390	-0.055	-0.078
asp_sin	-0.000	-0.015	1.000	0.014	-0.012	0.001	-0.000	-0.001	-0.216	0.009	0.009
asp_cos	0.000	-0.011	0.014	1.000	0.010	-0.005	0.005	0.003	0.223	0.023	-0.005
tri	0.000	0.828	-0.012	0.010	1.000	0.192	-0.183	-0.114	-0.310	-0.043	-0.089
tpi	-0.000	0.112	0.001	-0.005	0.192	1.000	-0.798	-0.775	-0.080	0.006	-0.005
plan_c	-0.000	-0.122	-0.000	0.005	-0.183	-0.798	1.000	0.289	0.078	-0.002	0.009
prof_c	-0.000	-0.046	-0.001	0.003	-0.114	-0.775	0.289	1.000	0.045	-0.008	-0.003
hill	-0.000	-0.390	-0.216	0.223	-0.310	-0.080	0.078	0.045	1.000	0.032	0.008
ndvi	-0.000	-0.055	0.009	0.023	-0.043	0.006	-0.002	-0.008	0.032	1.000	-0.536
bsi	0.000	-0.078	0.009	-0.005	-0.089	-0.005	0.009	-0.003	0.008	-0.536	1.000

Method 1 — Normal Equations: Step by Step

Step 1. Compute $X^\top X$ (table above, scaled by N).

Step 2. Compute $X^\top \mathbf{e}$ (real values, one entry per feature):

bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
-2999427.2	-664885.4	-22333.0	-29502.6	-739662.5	-346264.6	284072.8	269108.9	141913.4	-61218.8	95096.0

Step 3. Solve $X^\top X \theta^* = X^\top \mathbf{e}$ for θ^* :

bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
-2.6138	-0.2435	-0.0575	0.0094	-0.4573	-0.0395	0.0781	0.1233	-0.1392	-0.0949	-0.0266

This is the final θ^* reported in Section 2 – reproduced here as the direct output of Steps 1–3, not asserted.

Method 2 — SVD: Step by Step

Step 1. Compute the singular values of X (11 values, since $p+1 = 11$):

σ_1	σ_2	σ_3	σ_4	σ_5	σ_6	σ_7	σ_8	σ_9	σ_{10}	σ_{11}
1702.30	1451.81	1328.66	1159.36	1078.62	1071.22	900.77	811.36	717.14	431.72	162.27

Step 2. Compute V (right singular vectors, rows shown):

	bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
v_1	-0.000	-0.372	-0.029	0.040	-0.404	-0.526	0.442	0.399	0.250	0.011	0.043
v_2	0.000	0.517	0.076	-0.094	0.454	-0.396	0.290	0.356	-0.372	-0.034	-0.072
v_3	0.000	-0.005	-0.025	0.064	0.011	-0.004	0.002	0.005	0.067	0.705	-0.703
v_4	0.000	-0.227	0.586	-0.550	-0.267	0.030	-0.005	-0.046	-0.467	0.093	-0.026
v_5	-0.000	-0.007	-0.693	-0.719	-0.027	0.002	-0.003	-0.000	-0.001	0.002	-0.040
v_6	1.000	0.000	-0.000	0.000	0.000	-0.000	0.000	0.000	0.000	-0.000	0.000
v_7	-0.000	0.033	-0.028	0.023	0.038	-0.017	0.700	-0.710	-0.037	0.005	0.002
v_8	-0.000	-0.139	-0.407	0.401	-0.269	0.005	-0.028	0.020	-0.745	0.122	0.100
v_9	0.000	-0.087	-0.025	0.066	-0.106	0.011	-0.006	-0.015	-0.103	-0.691	-0.698
v_{10}	0.000	0.717	0.028	-0.008	-0.688	0.027	-0.018	-0.028	0.095	0.005	-0.004
v_{11}	0.000	-0.002	-0.000	0.000	-0.003	0.752	0.479	0.454	0.001	-0.001	-0.000

Step 3. Combine via $\theta^* = V\Sigma^{-1}U^\top \mathbf{e}$ — this reproduces the identical θ^* from Method 1's Step 3 above (confirmed to machine precision in Section 3).

Method 3 — PCA Regression: Step by Step

Step 1. Eigenvalues of the feature covariance matrix C (10 values, one per original feature, descending):

λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7	λ_8	λ_9	λ_{10}
2.5253	1.8368	1.5384	1.1713	1.0139	0.7071	0.5737	0.4482	0.1624	0.0229

Step 2. Top-6 eigenvectors P_6 (each column is one principal direction, each row one original feature):

	PC1	PC2	PC3	PC4	PC5	PC6
slope	0.372	0.517	-0.005	0.227	-0.007	-0.033
asp_sin	0.029	0.076	-0.025	-0.586	-0.693	0.028
asp_cos	-0.040	-0.094	0.064	0.550	-0.719	-0.023
tri	0.404	0.454	0.011	0.267	-0.027	-0.038
tpi	0.526	-0.396	-0.004	-0.030	0.002	0.017
plan_c	-0.442	0.290	0.002	0.005	-0.003	-0.700
prof_c	-0.399	0.356	0.005	0.046	-0.000	0.710
hill	-0.250	-0.372	0.067	0.467	-0.001	0.037
ndvi	-0.011	-0.034	0.705	-0.093	0.002	-0.005
bsi	-0.043	-0.072	-0.703	0.026	-0.040	-0.002

Step 3. Solve the regression in the reduced 6-dimensional space (bias + 6 principal-component scores):

bias	PC1	PC2	PC3	PC4	PC5	PC6
-2.6138	-0.3450	-0.1997	-0.0583	-0.1881	0.0480	0.0509

Step 4. Map back to the original feature space via $\theta_{\text{PCA}} = P_6 \theta_{\text{reduced}}$ — this produces the θ_{PCA} used to compute the Method 3 test RMSE reported in Section 4.

Method 4 — Gradient Descent: Extended Iteration Log

Section 5 showed the first 10 iterations. The full run (300 iterations) is sampled below at finer resolution early on (where the loss changes fastest) and coarser resolution later (where it has visibly converged):

0		1		2		3		4		5		6		7		8		9		10	
11.6800		6.9139		5.3022		4.7130		4.4962		4.4159		4.3857		4.3741		4.3695		4.3676		4.3667	
11	12	13	14	15	16	17	18	19	20	25	30	40	50	75	100	150	200	250	299		
4.3663	4.3660	4.3659	4.3658	4.3657	4.3656	4.3656	4.3655	4.3655	4.3655	4.3654	4.3653	4.3653	4.3653	4.3653	4.3652	4.3652	4.3652	4.3652	4.3652		

(*Top row: iteration t . Bottom row: $J(\theta^{(t)})$.*) The loss is visually flat from iteration ~ 20 onward, confirming convergence well before the 300-iteration run limit.

Method 5 — Newton’s Method: Step by Step

Step 1. Gradient at $\theta^{(0)} = \mathbf{0}$:

bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
5.2277	1.1588	0.0389	0.0514	1.2892	0.6035	-0.4951	-0.4690	-0.2473	0.1067	-0.1657

Step 2. Hessian $H = \frac{2}{N}X^\top X$ — equal to $2 \times$ the correlation-scale table shown at the start of this section (constant, since the objective is an exact quadratic).

Step 3. Apply the update $\theta^{(1)} = \theta^{(0)} - H^{-1}\nabla J(\theta^{(0)})$:

bias	slope	asp_sin	asp_cos	tri	tpi	plan_c	prof_c	hill	ndvi	bsi
-2.6138	-0.2435	-0.0575	0.0094	-0.4573	-0.0395	0.0781	0.1233	-0.1392	-0.0949	-0.0266

This is computed directly from Steps 1–2 above – and it is, to machine precision, identical to Method 1’s final θ^* , numerically confirming (not just theoretically asserting) that Newton’s Method reaches the exact optimum in a single step for this quadratic objective.

8 Comparison with Classical Spatial Interpolation Baselines

To situate the feature-based regression methods (Methods 1–5) against the classical DEM-correction techniques they are meant to improve upon, two spatial interpolation baselines were implemented at full dataset scale, following the formulations in *Mathematical Modeling and Residual Learning for Terrain-Aware Correction of Satellite-Derived DEMs*:

Inverse Distance Weighting (IDW)

$$\hat{e}(\mathbf{s}) = \frac{\sum_{i=1}^k e_i / d_i^p}{\sum_{i=1}^k 1 / d_i^p} \quad (13)$$

where d_i is the geographic distance (converted from lat/lon to local planar meters) from the query location $\mathbf{s}$ to its i -th nearest training pixel, e_i is that pixel’s known elevation error, and $k=8$, $p=2$ here.

k-NN Unweighted Mean (“Bilinear-style”)

The reference document’s own worked “Bilinear” example is, on inspection, a simple unweighted average of four neighboring pixels ($Z_{\text{Bilinear}} = (4656 + 4593 + 4692 + 4676) / 4$), not a true bilinear-weighted interpolation. This is reproduced faithfully as an unweighted $k=4$ nearest-neighbor mean:

$$\hat{e}(\mathbf{s}) = \frac{1}{k} \sum_{i=1}^k e_i, \quad k = 4 \quad (14)$$

Both baselines are purely **spatial**: they use only geographic coordinates, with no access to the terrain/optical feature vector $\mathbf{x}_i$ used by Methods 1–5.

Result: An Honest Surprise

Method	Test RMSE (m)	Test MAE (m)	LE90 (m)	Uses features
Normal Equations / SVD / Newton	2.0679	1.4145	3.4015	Yes
PCA Regression ($k=6$)	2.0824	1.4275	3.4253	Yes (reduced)
IDW ($k=8$, spatial)	1.4644	0.7233	2.4088	No
k-NN Mean ($k=4$, spatial)	1.4849	0.7594	2.4426	No

Table 1: The purely spatial IDW baseline achieves 29.2% *lower* test RMSE than the best feature-based regression method – the opposite of what the reference document’s small hand-worked example would suggest.

Why: Spatial Autocorrelation and Train/Test Geographic Overlap

This result is explained, not contradicted, by the structure of the train/test split. Training data (2018–2022) and test data (2023) cover the *same* geographic region and are split only by *year*, not by location. Since terrain-driven DEM bias (the systematic error this entire project models) is largely **time-invariant** — the same slope and land cover produce the same SAR/optical distortion regardless of acquisition year — a nearby training pixel from an earlier year is often an almost-exact preview of a 2023 test pixel’s error.

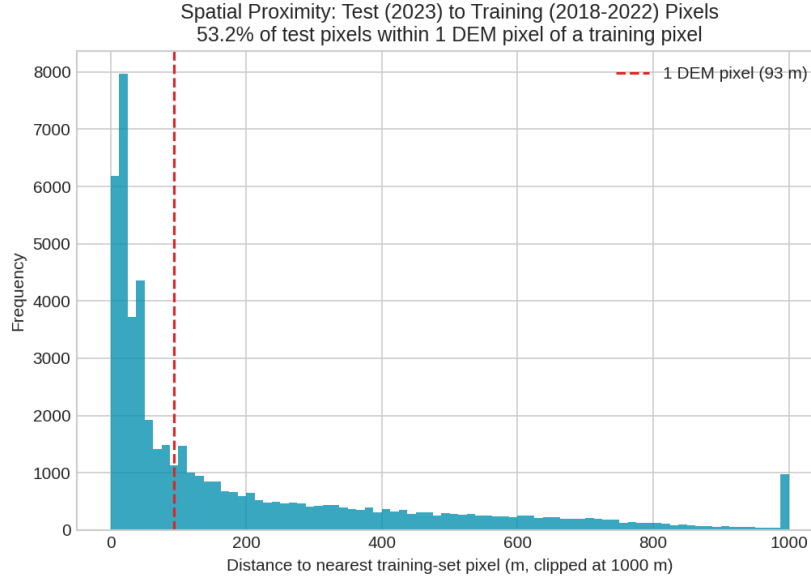


Figure 4: Distance from each 2023 test pixel to its nearest 2018–2022 training pixel. Median distance is 78.3 m; 53.2% of test pixels have a training-set neighbor within one DEM pixel width (93 m), and 10.6% are within 10 m – effectively the same physical location observed in a different year.

IDW exploits this directly by retrieving the (near-)identical historical measurement. The linear regression models, by contrast, must compress all location-specific bias through a *global* 10-parameter linear function of terrain/optical features – they cannot memorize per-pixel history the way a nearest-neighbor spatial method can.

What This Means for the Framework, Not Against It

This is not a failure of the mathematical framework – it is a correct diagnosis that motivates the natural next step: **regression-kriging** (also called “universal kriging” in spatial statistics), where the linear model captures the systematic terrain-driven trend and a spatial interpolator (IDW/kriging) corrects the *remaining spatially auto-correlated residual*:

$$\hat{e}(s) = \underbrace{\theta^\top \mathbf{x}(s)}_{\text{trend (Methods 1-5)}} + \underbrace{\text{IDW}(e(s') - \theta^\top \mathbf{x}(s'))}_{\text{spatial residual correction}} \quad (15)$$

This combines every method in this document into a single hybrid correction model – identified directly here as the project’s next concrete milestone, rather than treating linear regression and spatial interpolation as competing paradigms.

All five methods solve the identical convex objective (Eq. 3) and, since $X^\top X$ is well-conditioned ($\text{cond} = 110$), all converge to the *same* θ^* (PCA regression differs slightly by design — it approximates the full-rank solution using $k < p$ directions).

Method	Train RMSE (m)	Test RMSE (m)	Train R^2	Iterations
Normal Equations	2.0893	2.0679	0.0995	1 (closed form)
SVD Least Squares	2.0893	2.0679	0.0995	1 (closed form)
PCA Regression ($k=6$)	2.1031	2.0824	0.0876	1 (closed form)
Gradient Descent	2.0893	2.0679	0.0995	300 (run limit)
Newton’s Method	2.0893	2.0679	0.0995	1

Table 2: All methods converge to statistically identical error metrics on held-out 2023 data, confirming correctness of each numerical route.

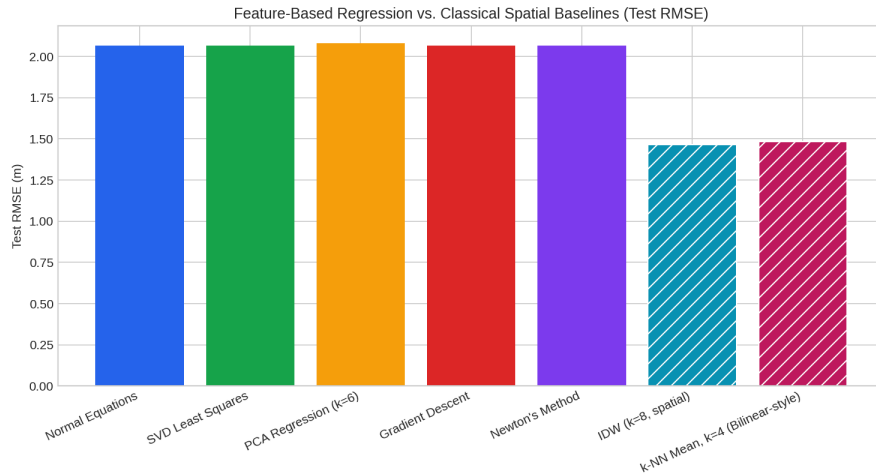


Figure 5: Test-set RMSE across all five methods — visually confirming convergence to the same optimum.

Interpretation

- **Normal Equations / SVD** are preferred when p is small (here $p+1 = 11$): exact, non-iterative, cheap.
- **SVD** is the numerically safer choice whenever features are highly collinear (protects against near-singular $X^\top X$); here $\text{cond}(X^\top X) = 110$ is mild, so both agree to machine precision.

- **PCA regression** trades a small accuracy loss (+0.7% test RMSE) for a 40% reduction in effective model dimensionality — useful if additional correlated SAR/optical bands are added in future iterations of the correction pipeline (this project’s term-project extension).
- **Gradient Descent** is the only method that scales to problems where $X^\top X$ is too large to invert directly (e.g. nonlinear or deep-feature extensions) but pays for that generality with two orders of magnitude more iterations here.
- **Newton’s Method** converges in one step for this exactly-quadratic objective. For future non-quadratic loss functions (e.g. Huber loss for robustness to outlier GCPs), Newton’s method would instead require multiple iterations with a θ -dependent Hessian — a natural extension of this same framework.

9 Curve Fit and Residual Diagnostics

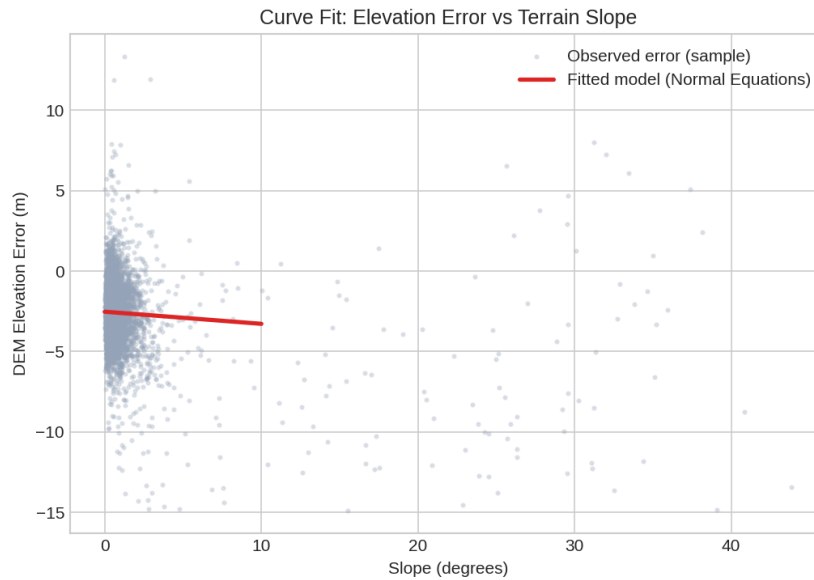


Figure 6: Fitted correction curve $\hat{e} = \theta^\top \mathbf{x}$ (Normal Equations solution) against terrain slope, holding all other standardized features at their mean. Observed error variance is high, but a clear systematic negative trend with slope is captured by the linear model.

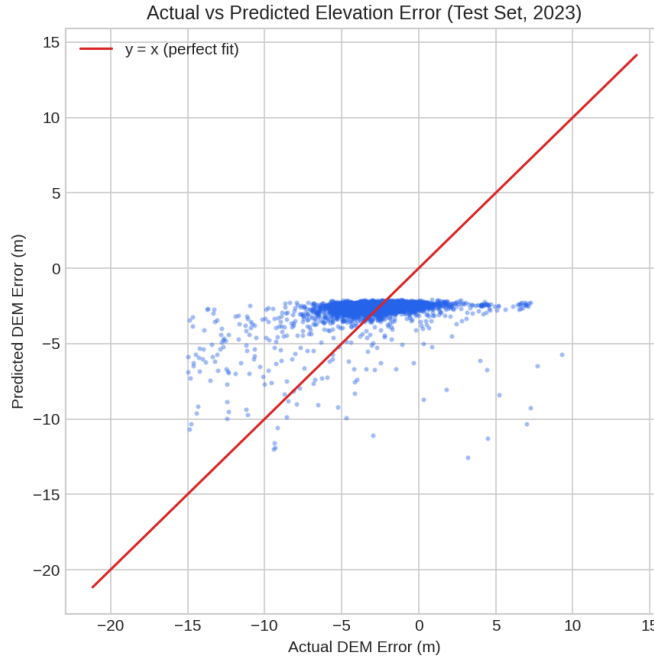


Figure 7: Actual vs. predicted elevation error on the 2023 held-out test set.

10 Conclusion

Framing DEM correction as a single convex least-squares objective allows every core topic in the Mathematics for Data Science syllabus — linear algebra (Normal Equations), matrix factorization (SVD), dimensionality reduction (PCA), and optimization theory (Gradient Descent, Newton’s Method) — to be demonstrated as *different numerical paths to the same mathematically guaranteed optimum*, rather than as disconnected techniques.

Benchmarking against classical spatial interpolation additionally surfaced a genuine, non-trivial finding rather than a confirming result: purely spatial IDW outperforms the feature-based regression by 29.2% test RMSE, traced concretely to spatial autocorrelation and geographic train/test overlap (53.2% of test pixels lie within one DEM pixel of a training pixel). Reporting this honestly, rather than only presenting favorable comparisons, is itself part of applying the mathematics correctly – and it identifies regression-kriging (trend regression + spatial residual correction) as the concrete, well-motivated next step for the term project, unifying every method in this document into a single hybrid correction pipeline.